

Checkable Lower Bounds for Convex Mixed-Integer Nonlinear Optimization through Rational Outer Approximations

Abstract

An exact mixed-integer linear programming (MILP) proof certifies a bound only for its own input. To certify a lower bound for a convex mixed-integer nonlinear program (MINLP) by outer approximation, two further links must be checked: every nonlinear cut must be valid for the stated model, and the proved MILP must be exactly the master built from those cuts and the original constraints. Building on established convex certificates, rigorous-bounding principles, and exact MILP proofs in the solver-independent VIPR format, we develop and implement a certificate interface that checks both links. One exact interpretation of the loaded expression tree governs curvature checks, cut checks, and master construction. Rational affine cuts are justified by rigorous function and derivative enclosures, with a rational intercept correction for bounded, half-line, and free coordinates. The regenerated master is matched to the proof input, and every discrete inference is replayed in rational arithmetic. We prove that accepted evidence bounds the objective at every feasible point of the interpreted model, independently of numerical subproblem accuracy and without attainment or Slater assumptions. Lean proofs cover the mathematical contract and include reference checkers for typed discrete proofs and rational quadratic matrices; the Python implementation and the benchmark certificates are not formally verified. On 289 convex MINLPLib models, a frozen uniform protocol yields certificates that pass separate replay for 203. Exact audits find three generated proofs that an external VIPR checker accepted although they contain invalid linear-combination or integer-disjunction steps, and show two saved solver points to be infeasible.

Keywords: convex MINLP; outer approximation; lower-bound certificates; rational arithmetic; verification.

1 Introduction

Exact mixed-integer linear programming (MILP) solvers can emit proofs in the VIPR format, whose inferences are checked in rational arithmetic independently of the solver [6, 14]. Such a proof certifies a bound for its own input model. For a convex mixed-integer nonlinear program (MINLP), outer approximation (OA) supplies the classical link to a MILP: affine underestimators of the convex functions define a MILP master whose lower bounds transfer to the original problem [9, 17]. Certifying an MINLP bound through this link requires two further checks. Every nonlinear cut must be valid for the stated nonlinear model, and the proved MILP must be exactly the master justified by those cuts and the original constraints. Without these checks, exact discrete arithmetic can certify the wrong relaxation. For example, rounding a tangent’s slope without correcting its intercept can exclude a feasible point (Example 3.4), and a proof for a different master cannot repair that error.

For minimization, we seek a rational number β and evidence establishing

$$\beta \leq f(x) \quad \text{for every feasible mixed-integer point } x \quad (1)$$

for a precisely interpreted nonlinear model. The evidence should be checkable without repeating the numerical search that produced it.

We develop and implement a certificate interface that performs both checks. One exact interpretation of the loaded expression tree governs domain and curvature checks, derivative and interval evaluation, and master construction. Rigorous enclosures at a supplied support point justify each rational affine cut. The checker regenerates the rational master, matches it to the input of a VIPR proof, and replays every discrete inference in rational arithmetic. Numerical search supplies candidate points, slopes, and proofs; acceptance depends only on their checked properties, not on the accuracy of numerical subproblem solutions.

1.1 Contribution and scope

The contribution is the integration of established convex-support and discrete-proof principles into a checked nonlinear-to-linear certificate interface, together with evidence about its reliability.

1. *Soundness.* Accepted evidence satisfying the stated contracts implies (1) (Theorem 3.7). The proof combines three parts: rational bound propagation preserves feasible points (Lemma 3.1); a rational cut test covers bounded, one-sided, and free coordinates (Theorem 3.2, Corollary 3.3); and an invariant for the admitted VIPR inferences gives an unconditional master bound (Proposition 3.5, Theorem 3.6). The argument maps every feasible point into the master with the same objective value. It needs no minimizer, Slater point, or finite optimum. An independently checked feasible point turns the bound into a certified optimality gap (Corollary 3.8).
2. *Implementation.* The checker recognizes curvature through sufficient composition rules and exact tests for quadratics, monomials, and one-variable linear fractions (Section 4.2). It checks cuts with outward interval arithmetic and exact endpoint extraction, binds the proof to the regenerated master under a checked variable bijection, and accepts complete proofs in a restricted subset of VIPR (Sections 4.3–4.5). It rejects two unsound behaviors reproduced in an inspected version of the external checker `viprchk` (Section 4.6). A suite of 161 targeted tests exercises the executable contracts and regression cases.
3. *Formal verification.* Lean proofs cover 49 mathematical obligations: propagation, curvature and cut rules, typed discrete checking, master identity, bound transfer, and primal completion. They include executable reference checkers for typed discrete proofs and rational quadratic matrices (Section 5). The nonlinear theorems take genuine derivative and enclosure data as hypotheses. The Python expression, interval, master-matching, and proof-checking software remains in the trusted base: it is not shown to refine the Lean checkers, and no benchmark certificate is checked in Lean.
4. *Experiments and audits.* On 289 convex MINLPLib models [21], a frozen uniform protocol with 30-second search budgets yields artifacts that pass separate replay for 203 models. Of the rest, 19 are rejected and 67 fail curvature or domain admission before a master is built (Section 6.1). Of the 203 bounds, 80 lie at or below an unverified library reference value, with normalized difference at most 10^{-2} . Exact audits (Section 6.2) find three newly generated proofs that the external checker accepted although they contain invalid linear-combination

or integer-disjunction steps, and show that two saved solver points are infeasible. A checked feasible point completes an exact optimality proof for `clay0204m`. Accepted uniform-run proofs total about 14.2 GB (Section 6.3).

5. *Reusable artifacts*. Portable checker code, frozen inputs and outcomes, complete proof bundles, exact failure extracts, and original-model witness audits support reproduction (Appendix A). A descriptive catalogue collects checked bounds for 222 distinct loaded models across all recorded protocols; it is a collection, not a uniform-run success rate.

We claim neither a new general support-minimization principle nor a certificate-size or solver-performance advantage over prior work.

The implementation covers supported smooth convex expressions and exactly recognized quadratic and structured functions. General nonconvex MINLP, nonlinear equalities, and unsupported expression domains lie outside its contract. It certifies the exact interpretation of the loaded expression model, whose relation to the source modeling file is specified in Section 2. Checked feasible points exist for `clay0204m` and a small quadratic example; proximity to a library reference value supplies no such point.

1.2 Relationship to prior work

Mixed-integer convex optimality certificates. Certificates for convex MINLP are established. Baes et al. [1] derive finite optimality certificates and mixed-integer duality using subgradients and mixed-integer-free polyhedra. Under their respective assumptions, their Theorems 1 and 3 bound the number of certificate points by $2^{|J|}$, where $|J|$ is the number of integer variables; this bounds the number of points, not the bit length of their real-valued data. Basu et al. [2] relate finite convex optimality certificates to Helly numbers for more general feasible sets.

The closest computational work is Halbig et al. [13]. Their generalized certificates consist of suspension points (points that anchor the certificate hyperplanes) and hyperplanes. They give construction and reduction algorithms, structural bounds on certificate size, and experiments on MINLPLib and MIPLIB. Their verification removes integrality and solves a continuous convex problem to test each certificate plane (Algorithm 3); a linear mixed-integer problem tests integer-freeness of the certificate polyhedron.

Our interface instead checks each cut by recomputing rigorous function and derivative enclosures at a supplied support point, then replays saved rational inferences for the discrete bound. The result is an OA certificate with a different checking procedure from the geometric certificates above. The certified bound may come from an interrupted master solve and need not equal a known optimum. Theorem 3.7 is a conditional validity result, not a completeness theorem guaranteeing such evidence for every optimum.

Outer approximation and numerical safeguards. Coey et al. [7] derive OA cuts from conic dual certificates and analyze scaling under MILP feasibility tolerances. We instead check affine underestimation for explicitly interpreted nonlinear expressions and export an exact rational master proof; the two approaches differ in their nonlinear representations and checking obligations.

Directed rounding and rational repair are established safeguards for approximate numerical information. Neumaier and Shcherbina [23] develop safe bounds and cuts for linear and mixed-integer linear programming, Eifler and Gleixner [10] combine safe Gomory mixed-integer cuts with rational proof verification, and Borst et al. [4] treat certified propagation and dual proof analysis in an exact MIP solver.

For nonlinear problems, Jansson [15] develops rigorous lower bounds for convex optimization, and Jansson [16] surveys verified convex and conic computations and their use in reliable global and combinatorial optimization. Messine and Trombettoni [20] minimize supporting affine functions over finite boxes to bound convex relaxations; our finite-box correction applies this calculation to the residual left after subtracting the proposed affine row. Elloumi et al. [11] describe QIBEX-R, which combines interval branch-and-bound for quadratic mixed-integer problems with curvature correction and rigorous relaxation bounds. These methods are precedents for our nonlinear layer. Our contribution joins that layer to explicit model and master binding and to replay of a saved discrete proof.

MILP proofs and checker correctness. Cheung et al. [6] introduce the VIPR format, whose inferences can be checked independently of the MILP solver that produced them. Exact MILP solving and VIPR proof production are available in SCIP [14], and recent work constructs VIPR proofs from a black-box ILP solver by rational reconstruction [25]. These methods supply discrete evidence; they do not establish the validity of nonlinear cuts supplied as input.

The semantics of a proof format and the correctness of a particular checker are separate issues. Wood et al. [27] prove a logical encoding of VIPR validity correct in Why3 and implement an SMT-based checker; their theorem covers the logical transformation, not the C++ translation program. They also identify trust-sensitive aspects of the existing checker. The CakeML project contains a VIPR formalization, a pure checker, and a verified program with theorems about its file and standard-input semantics [5]. Our Python replay kernel is a separate, unmechanized implementation for a restricted subset of VIPR. Its acceptance contract additionally binds the proof to the input master and rejects unsupported or malformed cases.

Formal nonlinear optimization and verified reductions. Solovyev and Hales [24] verify nonlinear inequalities with Taylor interval approximations in HOL Light. Narkawicz and Muñoz [22] verify a generic global-optimization branching algorithm in PVS. Magron et al. [19] combine semialgebraic approximations and sums-of-squares witnesses with Coq to prove nonlinear lower bounds. These are precedents for proof-producing numerical computation, with different representations of the nonlinear and discrete reasoning. CvxLean verifies optimization reductions in Lean [3]; its proofs show that a transformed problem represents the original one, while the external numerical solve is a separate obligation. The same distinction governs our formalization (Section 5).

Organization. Section 2 fixes the model and the certificate contract. Section 3 proves soundness. Section 4 describes the executable checker and Section 5 the Lean coverage. Section 6 reports the experiments and audits, and Section 7 concludes. Appendix A describes the accompanying materials, and Appendix B records the experimental protocols and full accounting.

2 Model semantics and the certification contract

2.1 The mathematical problem

Let $J \subseteq \{1, \dots, n\}$ index the integer variables. We consider

$$\begin{aligned} p^* = \inf \quad & f(x) \\ \text{subject to} \quad & x \in B_0, \quad x_j \in \mathbb{Z} \quad (j \in J), \\ & \ell \leq Ax \leq u, \\ & h_i(x) \leq 0 \quad (i = 1, \dots, m), \end{aligned} \tag{2}$$

where A is rational, each finite component of ℓ, u is rational, and $B_0 = \prod_{j=1}^n [L_j^0, U_j^0]$ has rational finite endpoints and may have infinite endpoints. Infinite endpoints have their usual half-line or whole-line meaning. Binary variables are integer variables restricted to $[0, 1]$, and a fixed variable has equal lower and upper bounds. The functions $f, h_1, \dots, h_m$ are defined, finite, and convex on B_0 ; this sufficient domain contract is deliberately stronger than definedness at feasible points only. Functions may include rational powers, exponentials, and logarithms; the coefficients in their expressions are rational, but their values need not be. Let F denote the feasible set of (2). An upper nonlinear row $g(x) \leq u_i$ is normalized as $h_i = g - u_i$; a lower row $g(x) \geq \ell_i$ is normalized as $h_i = \ell_i - g$. Convexity is required after normalization. Affine equalities are allowed through equal row bounds. General nonlinear equalities and two-sided nonlinear rows are rejected by the present implementation.

We use an infimum because a lower-bound certificate does not need attainment. The basic assertion is $\beta \leq f(x)$ for all $x \in F$. It remains meaningful when F is empty and, with the extended-real convention $\inf \emptyset = +\infty$, implies $\beta \leq p^*$. It does not certify that F is nonempty. Maximization of a concave objective $f_{\max}$ is treated by minimizing $-f_{\max}$; a lower bound β for that minimization gives the upper bound $-\beta$ in the original sense.

An epigraph variable t handles a nonlinear objective. The lifted feasible set is

$$\hat{F} = \{(x, t) : x \in F, f(x) - t \leq 0\}, \quad (3)$$

and the lifted objective is t . In particular, $(x, f(x)) \in \hat{F}$ for every $x \in F$, and no artificial finite bound on t is needed. For affine objectives, the master retains the exact affine objective, including its constant.

2.2 Expressions and model identity

A certificate refers to a particular expression model. A rational coefficient p/q , with integers p, q and $q > 0$, denotes its exact real value. In the implementation, every finite floating-point leaf of the loaded Pyomo tree denotes its exact stored binary value as a rational, and every integer leaf denotes that integer. All subsequent symbolic additions and multiplications use exact arithmetic. This rule also applies to bounds, fixed values, row shifts, exponents, and objective constants. Addition, multiplication, and powers are interpreted compositionally; logarithms and noninteger powers require their stated domains. Algebraic normalization is admissible only when it preserves this interpretation on the certified domain; in particular, cancelling a denominator must not remove a domain restriction.

Three objects must be distinguished: the source modeling file, the model built by the modeling environment, and the exact expression model that the certificate system checks. A source decimal such as 0.1 can become a binary floating-point value during construction. Treating that value as an exact rational is a precise convention, but it does not recover the decimal rational 1/10. Likewise, floating-point aggregation before extraction cannot be undone by storing the aggregate as a rational, and if the modeling environment rewrites $x/3$ with a binary approximation to 1/3, that coefficient defines the checked input. The Python model loader executes the input file; it is trusted input construction, not a sandbox for untrusted programs.

The distinction can matter for more than tiny changes in the optimum. If the binary values of 0.1, 0.2, and 0.30000000000000004 are treated as exact coefficients, then

$$(0.1)x^2 + (0.2)x^2 - (0.30000000000000004)x^2 = -2^{-55}x^2. \quad (4)$$

Floating-point coefficient aggregation can instead produce zero. The first expression is concave and the second affine, so a convexity check of one cannot justify underestimator evaluation of the other.

The required invariant is therefore that curvature analysis, domain checks, differentiation or subgradient justification, interval evaluation, and master construction all refer to the same exact expression model. A reproducible artifact must identify that model and document its relation to the source file; a hash identifies bytes but does not prove semantic equivalence. Since extraction is not formally verified, the extraction code and its interpretation rules belong to the trusted base. Agreement with a solver that reads another input format is an empirical comparison unless equivalence of the interpreted models has been established.

2.3 Boxes, support points, and domains

A checker may derive a tighter rational box $B \subseteq B_0$ from the affine rows and integer restrictions, provided it establishes $F \subseteq B$. Propagation need not reach a fixed point: every accepted tightening must preserve all feasible points, and incomplete propagation affects strength, not validity.

The implementation first validates original expression domains on B_0 , before algebraic cancellations, and does not infer missing domains from nonlinear constraints. For example, x/x cannot be replaced by 1 on a box containing zero. This can reject an otherwise valid constrained problem. Later linear bound propagation may strengthen a cut by reducing B without changing this input-domain convention.

A cut for a nonlinear row h_i is an affine function $a^\top x + b$ satisfying

$$a^\top x + b \leq h_i(x) \quad (x \in B). \quad (5)$$

Here $a \in \mathbb{Q}^n$ and $b \in \mathbb{Q}$. Condition (5) is sufficient, but not necessary, for $a^\top x + b \leq 0$ to be valid for the row $h_i(x) \leq 0$ on B ; cut validity is not equivalent to underestimation on the whole box.

A point $z \in B$ used to justify an underestimator need not satisfy the nonlinear rows or integrality constraints. What is required is a valid support inequality

$$h_i(x) \geq h_i(z) + s^\top (x - z) \quad (x \in B) \quad (6)$$

for a vector s , and justified enclosures of the quantities used in the rational correction. Differentiability of a convex function on an open convex neighborhood of B is one sufficient condition, with $s = \nabla h_i(z)$. More general support conditions can apply at boundary points, but convexity alone does not justify evaluating an undefined derivative there: $-\sqrt{x}$ is convex and finite on $[0, 1]$ but has no finite supporting slope at zero. The implementation must reject a proposed cut when its evaluation or support conditions cannot be established. The mathematical formulation permits subgradients; the implementation does not support arbitrary nonsmooth functions.

2.4 What a complete check establishes

A rational master M retains justified bounds, affine rows, integrality, and valid nonlinear-row cuts, together with either the original affine objective or epigraph cuts. It must contain an image of every feasible point of (2) with the same objective value; a proof of the master’s bound then transfers to (1). The master objective includes the affine constant exactly once. Variable correspondence, row coefficients and directions, bounds, integrality, and the objective sense must match the proof input. A proof for a presolved model requires a separately justified transformation; matching only its objective value is insufficient. The implication does not assume accurate nonlinear subproblem solutions.

We distinguish three levels of checking when we use the term “verified”.

1. *Nonlinear-lemma checking* establishes curvature, domains, bound propagation, and the validity of the rational cuts. It does not establish a master bound.

2. *Complete certificate checking* additionally checks master identity and a proof of the reported master bound under an explicitly stated proof-checker contract.
3. *Proof-assistant verification* establishes the specific theorems or executable checks processed by the proof assistant. Its coverage is stated separately from that of the software checker.

Even complete certificate checking has a trusted computing base: the arithmetic and expression implementation, parser, proof checker, and their execution environment, unless individually verified. Replay does not repeat the producer’s numerical search, but it shares code for mathematical rules with the producer, so it is not an independent development in every component. We therefore describe it as a replay separate from the numerical search, with shared trusted components identified explicitly.

A complete lower-bound check returns a finite bound only after all its obligations pass. Partial nonlinear and master checking has a distinct result and yields no certified bound. Rejection can reflect unsupported syntax, a failed sufficient condition, or a mathematically invalid proof step; these outcomes have different diagnostic meanings, and none is treated as acceptance.

Finally, a feasible solution of the MILP master is generally not feasible for the nonlinear model. A primal witness must independently satisfy all original domains, bounds, integrality conditions, affine rows, and nonlinear rows, and come with an outward objective enclosure. For minimization, such a witness with objective upper bound U and a checked lower bound β prove $0 \leq U - p^* \leq U - \beta$. Equality $U = \beta$ proves attainment and exact optimality. Comparison with a rounded reference objective supplies none of these feasibility premises.

3 Soundness of the certificate interface

This section proves the implications used by the checker. The results apply to accepted evidence satisfying their hypotheses; they do not assert that every convex MINLP has evidence accepted by the implementation. We treat the nonlinear support calculation, the discrete proof, and their composition separately, so that a successful check of one cannot hide a defect in another.

3.1 Rational propagation preserves feasible points

Start with the declared box, intersect binary coordinates with $[0, 1]$, and substitute exact fixed values. For a row $\sum_j c_j x_j \leq u$, let R_k^- be a lower bound on $\sum_{j \neq k} c_j x_j$ over the current box. If $c_k \neq 0$ and R_k^- is finite, the implied bound is

$$x_k \leq (u - R_k^-)/c_k \quad (c_k > 0), \quad x_k \geq (u - R_k^-)/c_k \quad (c_k < 0). \quad (7)$$

For a lower row $\sum_j c_j x_j \geq \ell$, use an upper bound R_k^+ for the other terms and divide $\ell - R_k^+$ by c_k , reversing the corresponding directions. The coefficient sign determines which variable endpoint bounds each term. A missing endpoint gives no finite tightening in that direction. All finite operations are rational. An upper bound on an integer coordinate may be rounded down, and a lower bound rounded up.

Lemma 3.1 (Feasible-point preservation). *Any finite sequence of these operations produces a box B containing every original mixed-integer feasible point. If a coordinate interval becomes empty, the original feasible set is empty.*

Proof. Initially every feasible point satisfies the declared, fixed, and binary bounds. Suppose it lies in the box before an update. Its other terms are at least R_k^- , so the upper row implies $c_k x_k \leq u - R_k^-$.

Division gives (7). The lower-row argument is identical with an upper bound on the other terms. Integer rounding preserves each feasible integral coordinate. Intersecting with an implied bound therefore preserves the point. Induction proves the inclusion; an empty coordinate interval cannot contain such a point. $\square$

The implementation makes at most 20 sweeps; this limits strength, not soundness. Bounds from integer rounding need not contain the continuous relaxation, since the inclusion needed here is $F \subseteq B$. Every cut is nevertheless justified on the whole resulting box. A constant affine row is either a tautology or a contradiction. The finite-bound interface omits the former and rejects the latter; it does not issue a nonlinear infeasibility certificate merely because propagation detects a contradiction.

3.2 Safe rational affine underestimators

The finite-box calculation below is support minimization applied to an affine residual, an established rigorous-bounding principle [15, 20]. We state it with explicit rational-enclosure and unbounded-coordinate conditions because these conditions are what replay checks. Write a normalized row, including an objective epigraph row if necessary, as

$$r(x) = g(x) + q^\top x + c. \quad (8)$$

Here q, c are rational and g is convex on the certified box. Let $z \in B$ and suppose a finite vector p satisfies

$$g(x) \geq g(z) + p^\top (x - z) \quad (x \in B). \quad (9)$$

A justified subgradient gives this inequality. The implementation uses finite symbolic derivatives in supported differentiable cases and has no general subgradient oracle. The supplied point z and cut slope a must be rational; the true values $g(z)$ and p need not be.

Set $\phi(z) = r(z) - a^\top z$ and $d = p + q - a$. For each coordinate define

$$W_j(d_j) = \sup_{y \in B_j} d_j(z_j - y). \quad (10)$$

Because $z_j \in B_j$, $W_j \geq 0$. Table 1 gives the exact formulas and finiteness conditions. A sign failure on a half-line, or a nonzero residual slope on a free coordinate, makes this particular support bound infinite; the nonlinear function may still have a finite affine underestimator.

Table 1: Exact coordinate correction, with $z_j \in B_j$. Infinite endpoints are not used in arithmetic.

B_j	Condition for finite W_j	$W_j(d_j)$
$[L_j, U_j]$	L_j, U_j finite	$\max\{d_j(z_j - L_j), d_j(z_j - U_j)\}$
$[L_j, +\infty)$	$d_j \geq 0$	$d_j(z_j - L_j)$
$(-\infty, U_j]$	$d_j \leq 0$	$d_j(z_j - U_j)$
$\mathbb{R}$	$d_j = 0$	0

Theorem 3.2 (Rational cut safety). *Let B be nonempty, $z \in B$, and suppose (9) holds. If every W_j is finite and $b \in \mathbb{Q}$ satisfies*

$$b \leq \phi(z) - \sum_j W_j(d_j), \quad (11)$$

then $a^\top x + b \leq r(x)$ for every $x \in B$. In particular, $a^\top x + b \leq 0$ is valid for every point in B satisfying $r(x) \leq 0$.

Proof. Adding the affine terms to (9) gives

$$r(x) - a^\top x \geq \phi(z) + \sum_j d_j(x_j - z_j) \geq \phi(z) - \sum_j W_j(d_j) \geq b.$$

The middle inequality is the definition of each supremum. On a finite interval its value is the larger endpoint value; on a half-line it is finite exactly for the listed sign; on $\mathbb{R}$ only zero slope is bounded above. This also proves Table 1. $\square$

Corollary 3.3 (Finite rational enclosure test). *Suppose $v \in \mathbb{Q}$ satisfies $v \leq \phi(z)$ and finite rational intervals $[d_j^-, d_j^+]$ enclose d_j . The conditions and quantities in Table 2 imply $W_j(d_j) \leq \widehat{W}_j$. Therefore*

$$b \leq v - \sum_j \widehat{W}_j \quad (12)$$

certifies the cut by Theorem 3.2.

Proof. On a finite interval, $z_j - L_j \geq 0$ and $z_j - U_j \leq 0$. The respective products increase with d_j and decrease with d_j , giving the two endpoint bounds. The same signs give the half-line bounds, and the sign tests establish finiteness. On a free coordinate the enclosing interval must be exactly $[0, 0]$. Finally $v \leq \phi(z)$ and $\widehat{W}_j \geq W_j$ imply (11). $\square$

Table 2: Rational sufficient tests for interval gradient data. All intervals must be nonempty and finite.

B_j	Acceptance condition	$\widehat{W}_j$
$[L_j, U_j]$	$d_j^- \leq d_j^+$	$\max\{d_j^+(z_j - L_j), d_j^-(z_j - U_j)\}$
$[L_j, +\infty)$	$d_j^- \geq 0$	$d_j^+(z_j - L_j)$
$(-\infty, U_j]$	$d_j^+ \leq 0$	$d_j^-(z_j - U_j)$
$\mathbb{R}$	$d_j^- = d_j^+ = 0$	0

For a fixed coordinate, $L_j = U_j = z_j$, so the correction is zero. Exact elimination of a fixed variable avoids differentiating it; without elimination, its support data must still be justified. Every variable in the row participates, including variables occurring only in $q^\top x$. An absent coefficient in the sparse cut means $a_j = 0$, not an absent residual term.

Direct outward interval evaluation of the right-hand side of (11) is also a valid sufficient test. The implementation uses this variant, with rational input conversion and exact extraction of the final interval's lower endpoint. Dependency loss in interval arithmetic can weaken the cut but cannot invalidate an actual enclosure. On an upper-unbounded coordinate the producer can choose a_j below a rigorous lower bound for $p_j + q_j$; on a lower-unbounded coordinate it can choose it above a rigorous upper bound. Free coordinates require an exact residual zero. The epigraph coordinate has coefficient -1 and zero nonlinear derivative, so taking its slope as -1 meets that condition without bounding the epigraph variable.

Example 3.4 (Why the intercept must change when the slope is rounded). Let $r(x) = x^2 - 1/9$ on $[0, 1]$ and $z = 1/3$. Its tangent is $(2/3)x - 2/9$. Replacing the slope by $a = 67/100$ while keeping the intercept $-2/9$ makes the cut left-hand side equal to $1/900$ at the feasible point z , so rounding excludes an original feasible point. Here $d = 2/3 - 67/100 = -1/300$, $\phi(z) = -67/300$, and

$$W = \max\{-1/900, 1/450\} = 1/450.$$

The safe intercept $b = -203/900$ follows from (11). Indeed,

$$r(x) - ((67/100)x - 203/900) = (x - 67/200)^2 + 799/360000 > 0.$$

The largest intercept for this fixed slope is $-1/9 - (67/100)^2/4$, so the support correction is sufficient without being best possible. If the box were instead $[0, +\infty)$, the slope $67/100$ would fail the residual-sign test. Choosing $a = 3/5$ gives $d = 1/15$, $W = 1/45$, and the valid half-line intercept $b = -2/9$. Thus bounded and unbounded coordinates require different rounding decisions. Neither construction requires z to be an optimizer or an integer point.

Underestimation on the whole box is stronger than a feasible-set cut needs: on $[-1, 1]$ the row $x \leq 0$ implies $2x \leq 0$, although $2x \leq x$ fails for positive x . The interface deliberately uses the stronger condition because its evidence is simple to replay.

3.3 What the discrete proof must establish

Let M denote the rational mixed-integer linear master, and let $c_M^\top y$ be its linear objective. An objective constant is represented, if needed, by a separate variable fixed to one. We give the argument for minimization. A row is an equality or a weak linear inequality with rational coefficients. A row *dominates* another when it logically implies it by one of the checker’s admitted tests: the same left-hand coefficients with a stronger right-hand side and compatible direction, or a constant contradiction. Equalities can dominate either corresponding weak inequality.

A derived row C carries a finite set $A(C)$ of assumption-row indices. Suppose every supplied solution is exactly master feasible, and let b_M be the least objective value of those solutions when any are supplied. Define

$$S = \begin{cases} \{y \in M : c_M^\top y \leq b_M\}, & \text{if a solution is supplied,} \\ M, & \text{otherwise.} \end{cases} \quad (13)$$

A solution cutoff is valid on S , not necessarily on all of M . The following rules, which follow the standard VIPR semantics of solutions and assumptions [6, 27], respect this distinction.

Proposition 3.5 (Inference invariant). *Suppose every inference reference points to a strictly earlier row. For every row derived by the rules below, every $y \in S$ satisfying all assumptions in $A(C)$ satisfies C .*

1. *Original master rows have no assumptions. An assumption row has its own index as its assumption set.*
2. *A solution row must be dominated by $c_M^\top y \leq b_M$ and requires a supplied feasible solution. Its assumption set is empty relative to S .*
3. *A linear combination uses rational multipliers whose nonzero inequality terms all have the same direction after multiplication; equality terms are unrestricted. Its sum must dominate the claimed row. Its assumptions are the union from all nonzero terms.*
4. *A rounding step first forms such a combination. Every nonzero left-hand coefficient must be integral and multiply an integer variable. An upper right-hand side is rounded down, or a lower right-hand side up. The rounded row must dominate the claim. Equality rounding is not admitted.*

5. An unsplit step references two actual assumption rows D_1, D_2 of the form $a^\top y \leq k$ and $a^\top y \geq k+1$, in either order, where k and the linear form are integral on every master-feasible point. It also references branch conclusions C_1, C_2 that each dominate C . Its assumption set is

$$A(C) = (A(C_1) \setminus \{D_1\}) \cup (A(C_2) \setminus \{D_2\}).$$

Here the notation $\{D_i\}$ means the index of that assumption row.

Proof. Proceed in derivation order. Original rows and assumptions satisfy the invariant immediately. Solution rows hold by (13). For a combination, a point satisfying the union of dependencies satisfies every nonzero source row. Multiplication with the checked signs and addition give the summed relation, hence its dominated claim. Zero multipliers contribute neither an inequality nor an assumption. For rounding, the left-hand side is an integer at the point, so floor or ceiling of the right-hand side preserves validity.

For an unsplit step, take $y \in S$ satisfying $A(C)$. The integral disjunction is exhaustive, so y satisfies at least one D_i . Together with $A(C_i) \setminus \{D_i\}$, this gives every dependency of C_i . The induction hypothesis proves C_i , which implies C . Dependencies involving the other branch or any unrelated assumption remain in the displayed union; discharging one branch cannot erase them from the other branch's proof. $\square$

Theorem 3.6 (Unconditional master bound). *Suppose all solutions and every derivation pass the preceding rules, and some assumption-free derived row dominates $c_M^\top y \geq \beta$ for finite β . Then $c_M^\top y \geq \beta$ for every $y \in M$. No attainment assumption is required. With no supplied solutions, an assumption-free derived contradiction instead proves $M = \emptyset$.*

Proof. Proposition 3.5 proves the bound on S . With no supplied solutions, $S = M$. Otherwise a supplied solution attaining the finite minimum b_M among the finite list belongs to S , so $b_M \geq \beta$. Every point of $M \setminus S$ has objective greater than b_M , hence at least β . The contradiction statement follows from the same invariant with $S = M$. $\square$

Maximization uses the greatest checked solution value, a cutoff $c_M^\top y \geq b_M$, and a final upper inequality; the same proof applies with the objective order reversed. An infeasibility claim requires an empty solution section, so a cutoff cannot manufacture infeasibility by excluding feasible solutions. The stronger integer-objective cutoff $b_M - 1$ is outside the admitted rules. A valid earlier bound can suffice even when later valid rows are weaker or have unused assumptions, but every later row and the end of the file must still be checked. The argument establishes the semantic statement that the nonlinear layer needs, rather than matching a numerical endpoint in a proof header.

3.4 Transfer and primal completion

Let $s = 1$ for an original minimization and $s = -1$ for an original maximization, and set $h = sf_{\text{orig}}$. Use the exact affine objective if h is affine. Otherwise append a free real coordinate t , minimize t , and treat $h(x) - t \leq 0$ as another convex row. The master includes original affine rows, justified bounds, the original integrality conditions, and accepted cuts. Its extra constant variable, if present, is fixed to one. It has no other constraints without a separate validity justification.

Theorem 3.7 (Nonlinear bound transfer). *Suppose the box contains F , every master cut satisfies Theorem 3.2 on its applicable box, the proof input is the stated master under a checked variable bijection, and Theorem 3.6 establishes its bound β . Then*

$$sf_{\text{orig}}(x) \geq \beta \quad (x \in F).$$

Thus β is a lower bound for original minimization, and $-\beta$ an upper bound for original maximization. A verified infeasible master would imply $F = \emptyset$.

Proof. For $x \in F$, define $E(x)$ by preserving every original variable, setting $t = h(x)$ if needed, and setting the constant variable to one if present. Original affine rows, integrality, and propagated bounds hold by construction and Lemma 3.1. Every nonlinear row is nonpositive at x , and every epigraph row is zero at $E(x)$; safe underestimation therefore proves all master cuts there. Thus $E(x) \in M$. Its master objective equals $h(x)$, including the affine constant exactly once. The master bound applied to $E(x)$ proves the result. If M were empty, no such extension of a feasible x could exist. $\square$

For minimization this proves $\beta \leq \inf_{x \in F} f(x)$, with the convention $\inf \emptyset = +\infty$. The proof does not require a minimizer, a Slater point, or a finite optimum before the certificate is supplied. An unbounded relaxation or an unsuccessful check provides no finite bound. The infeasibility implication is broader than the implemented public API, which reports finite bounds only.

Corollary 3.8 (Primal completion). *Suppose an independently checked $\bar{x} \in F$ has a justified finite objective upper enclosure $f(\bar{x}) \leq U$ for minimization. A checked lower bound β proves*

$$\beta \leq p^* \leq f(\bar{x}) \leq U, \quad 0 \leq f(\bar{x}) - p^* \leq U - \beta.$$

If $U = \beta$, then $\bar{x}$ attains the exact optimum. The corresponding assertion for maximization follows by sign normalization.

Proof. Feasibility gives $p^* \leq f(\bar{x})$ and Theorem 3.7 gives the other lower bound. The two inequalities give the gap and, in the equality case, attainment. $\square$

The primal check must include variable bounds and integrality as well as rows and objective evaluation. A master solution supplies only the premise used in (13), and a recorded nonlinear reference value supplies neither a feasible point nor a rigorous objective enclosure.

4 Implementation and its checking boundary

The implementation separates a numerical producer from replay. The producer uses nonlinear and MILP solvers to seek useful evidence. Replay reconstructs the exact model, checks the cuts, matches the rational master, and checks its discrete derivations, without using the producer’s search history. The two share several mathematical utilities, so replay is independent of the numerical search but is not a separate implementation of every mathematical rule.

4.1 Artifact and execution contract

A certificate bundle consists of the supplied model, a nonlinear-lemma file, a rational LP master, and a completed VIPR proof. Each nonlinear lemma names its normalized row and supplies a rational point z , a sparse rational slope vector a , and an intercept b . The checker reconstructs the box and expressions from the model; it does not trust a supplied box, derivative value, curvature label, or reported solver bound. New bundles record the semantic identifier `pyomo-expression-rationals-v1` and a source hash. These fields may be absent in saved historical files; when present, the identifier must match and the hash is checked. In either case the model passed to replay defines the theorem being checked, and the frozen campaign manifest identifies its exact bytes.

Complete checking performs the following operations in order.

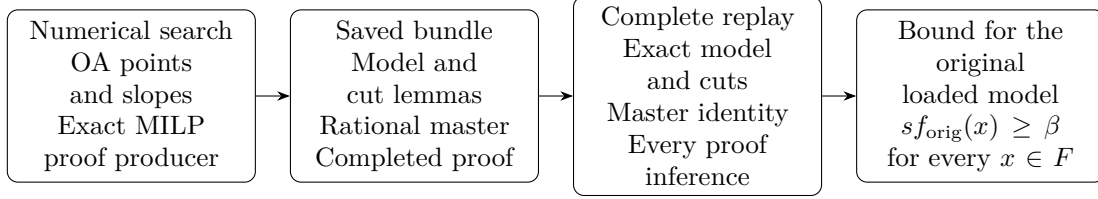


Figure 1: The certificate interface. Complete replay requires every checking obligation to pass; the last implication is objective-preserving bound transfer. Search is outside the trusted base once its output passes. Lean verifies the mathematical contract and typed reference checkers, with no verified connection to the Python implementation or benchmark certificates.

1. Load and validate the model, normalize the objective and nonlinear rows, check domains and curvature, extract exact affine rows, and propagate rational bounds.
2. Reconstruct each lemma’s row and symbolic derivatives, verify its point and variable names, and recompute a rigorous intercept bound. Require the supplied b to be no larger than that bound.
3. Regenerate the LP master and require byte equality with the saved LP file. Compare the VIPR problem against the reconstructed rational master semantically, as specified below.
4. Check every supplied master solution and every discrete proof step. Require a finite minimization lower bound for the normalized master. If an external checker was requested, require matching successful corroboration as well.

Only then does the public routine return `ok=True` and a bound in the original objective sense. Partial mode returns `status="partial"`, possibly with `partial_ok=True`, but `ok=False` and no certified bound. A missing proof, exception, unsupported input, or failed check cannot yield a certified numerical result. The lower-level VIPR kernel supports infeasibility and either objective sense, but the MINLP interface deliberately reports only finite normalized lower bounds; the infeasibility theorems of Section 3 do not enlarge this API.

The model loader executes a Python/Pyomo file and applies a documented dedenting repair to top-level model assignments in converted files; it therefore belongs to trusted input construction. Replay requires one active objective and refuses active disjunctive, logical, or special ordered set (SOS) constraints. Supported variable and affine-row names are a letter or underscore followed by letters, digits, or underscores; reserved auxiliary names cannot alias original variables. The exact extractor replaces fixed variables by their exact values and performs polynomial coefficient accumulation with rational arithmetic. Nonpolynomial subtrees remain available for original-domain checks, even when multiplied by zero. Differentiation and interval evaluation use the same rational-leaf interpretation, not a separate floating-point coefficient aggregation.

4.2 Sufficient curvature and domain rules

Curvature checking combines composition rules in the spirit of disciplined convex programming [12] with a small number of algebraic recognizers. Each expression carries a curvature classification and, where justified, rational lower and upper range bounds. These are separate facts: a product may be nonnegative without being convex or concave. Unknown curvature or a missing range endpoint is never replaced by a guess.

The composition principle follows directly from Jensen’s inequality. If v is convex and u is convex and nondecreasing on the range of v , apply monotonicity to $v(\lambda x + (1 - \lambda)y) \leq \lambda v(x) + (1 - \lambda)v(y)$ and then convexity of u . If u is convex and nonincreasing, use a concave v instead. Reversing both convexity inequalities gives the corresponding concave rules. The implementation applies these principles to sums, rational scalings, affine absolute values, and the functions in Table 3. Negative scaling interchanges convex and concave classifications.

Table 3: Principal scalar composition tests. “Positive” means a strictly positive certified lower bound; all original expression domains must also pass.

Expression	Sufficient condition	Curvature
$\exp(v)$	v affine or convex	Convex
$\log(v)$	v positive, affine or concave	Concave
$\sqrt{v}$	v nonnegative, affine or concave	Concave
$v^p, p \geq 1$	v nonnegative, affine or convex	Convex
$v^p, 0 < p < 1$	v nonnegative, affine or concave	Concave
$v^p, p < 0$	v positive, affine or concave	Convex
$v^{2k}, k \in \mathbb{Z}_{>0}$	v affine, or nonnegative convex, or nonpositive concave	Convex
$k_0/v, k_0 > 0$	v positive, affine or concave	Convex
$k_0^v, k_0 > 0$	v affine; also v convex if $k_0 > 1$, or concave if $k_0 < 1$	Convex

The power and reciprocal tests follow from first and second derivatives on the stated intervals; for k_0^v use $\exp(v \log k_0)$. Power one preserves curvature, and power zero is the constant one after the original base domain is checked. Nonnegative-domain rules extend to an included zero endpoint only when the expression is defined and continuous there; recognizing convexity at such an endpoint does not establish differentiability there.

Quadratic forms and norms. For $q(x) = x^\top Qx + b^\top x + c$ with rational symmetric Q , global convexity on $\mathbb{R}^n$ is equivalent to $Q \succeq 0$ because $\nabla^2 q = 2Q$. This gives a sufficient test on any certified box, including one with fixed coordinates. Off-diagonal monomial coefficients are split equally between Q_{ij} and Q_{ji} . The exact elimination test uses the following elementary characterization.

Lemma 4.1 (Exact quadratic tests). *For symmetric Q , a positive leading pivot α in $Q = \begin{bmatrix} \alpha & v^\top \\ v & C \end{bmatrix}$ reduces positive semidefiniteness to $C - vv^\top/\alpha \succeq 0$. A negative diagonal entry rules it out. A zero diagonal entry requires its whole remaining row and column to be zero, in which case they can be removed. These rules give an exact rational test. Moreover, if*

$$H = \begin{bmatrix} Q & b/2 \\ b^\top/2 & c \end{bmatrix} \succeq 0,$$

then $\sqrt{q(x)}$ is globally defined and convex.

Proof. For $\alpha > 0$, completing the square writes the quadratic form as $\alpha(t + v^\top z/\alpha)^2 + z^\top (C - vv^\top/\alpha)z$. A negative diagonal gives a negative value on a coordinate vector. If a diagonal is zero and some corresponding entry v_j is nonzero, the restriction to those two coordinates has a term $2v_j t$ with no t^2 term and takes negative values. This proves the elimination rules by induction. For the second statement, factor the real positive semidefinite matrix as $H = R^\top R$. Then $\sqrt{q(x)} = \|R(x, 1)\|_2$, the norm of an affine map, and the triangle inequality proves convexity. The factor need not be rational or computed by the checker; exact testing of H is sufficient. $\square$

Testing $-Q$ gives concavity. The homogenized test is only a sufficient rule for square roots of quadratics; it does not recognize every convex square-root expression. At a norm origin, finite subgradients may exist even though the symbolic-gradient route abstains.

Monomials and geometric means. The algebraic recognizer collects rational exponents in $m(x) = \prod_i x_i^{\alpha_i}$ on a positive orthant. It accepts convexity when all $\alpha_i \leq 0$, or when exactly one exponent is positive and $\sum_i \alpha_i \geq 1$. It accepts concavity when all $\alpha_i \geq 0$ and $\sum_i \alpha_i \leq 1$. A rational coefficient scales or reverses the result. These established conditions are stated by Lundell and Westerlund [18, Theorems 2.1–2.2, p. 507], who attribute them to earlier work. We include a direct proof to keep the recognition contract self-contained and free of floating-point eigenvalue calculations.

Lemma 4.2 (Monomial tests). *The preceding sufficient curvature tests hold on the positive orthant. When all nonzero exponents are positive, the resulting convexity or concavity extends continuously to the nonnegative orthant wherever the product is so defined.*

Proof. With $D = \text{diag}(1/x_i)$, differentiation gives

$$\nabla^2 m(x) = m(x) D(\alpha \alpha^\top - \text{diag } \alpha) D.$$

If every $\alpha_i \leq 0$, the middle matrix is the sum of a positive semidefinite outer product and a nonnegative diagonal matrix. For nonnegative exponents, weighted Cauchy–Schwarz yields $(\sum_i \alpha_i u_i)^2 \leq (\sum_i \alpha_i) \sum_i \alpha_i u_i^2$, so the middle matrix is negative semidefinite if the exponent sum is at most one.

For the remaining convex case remove zero exponents and write $\alpha = (p, -b_1, \dots, -b_k)$ with $b_i > 0$, $B = \sum_i b_i$, and $p \geq 1 + B$. The lower block of the middle matrix is $\text{diag}(b) + bb^\top \succ 0$, and $b^\top (\text{diag}(b) + bb^\top)^{-1} b = B/(1 + B)$, as follows by multiplying that matrix by $\mathbf{1}/(1 + B)$. Its Schur complement is $p(p - 1) - p^2 B/(1 + B) = p(p - 1 - B)/(1 + B) \geq 0$. If $k = 0$, this is simply $p(p - 1) \geq 0$. The Hessian criteria prove the interior assertions. Taking limits of the convexity inequalities proves their continuous boundary extensions. $\square$

Before distributing exponents, the syntax recognizer requires the factors of a fractional power of a product to be justified nonnegative variables and a unit constant. Original denominator and power domains are checked first, so identities such as $\sqrt{x^2} = x$ are not used on a negative domain. For example, the product-range calculation proves $xy \geq 0$ for $x, y \geq 0$, after which the exponent test recognizes $\sqrt{xy}$ as concave; the bilinear product itself is never labeled concave for that reason.

One-variable linear fractions. A further rule recognizes $(\alpha x + \beta)/(\gamma x + \delta)$ when the denominator is bounded strictly away from zero with a constant sign. For $\gamma \neq 0$, write

$$\frac{\alpha x + \beta}{\gamma x + \delta} = \frac{\alpha}{\gamma} + \frac{k}{\gamma x + \delta}, \quad k = \beta - \frac{\alpha \delta}{\gamma}.$$

Its second derivative is $2k\gamma^2/(\gamma x + \delta)^3$. Convexity therefore holds when k and the denominator have the same sign, concavity when their signs differ, and the function is constant if $k = 0$. A constant denominator uses rational scaling instead. This is a one-variable test, not a general curvature test for ratios or perspectives. An earlier perspective shortcut was disabled because curvature of the inner expression on its own variable bounds need not justify curvature on the actual ratio domain.

Domain and range enforcement. Range propagation uses rational endpoint arithmetic for sums, scales, integer powers, and finite products. Finite products use all four endpoint products. On sign-certified half-lines the corresponding extreme products give the finite endpoint, and unsupported sign combinations return an unknown range. Exponential, logarithmic, and rational-power endpoint bounds use outward interval arithmetic at 128-bit precision, with exact rational extraction of their dyadic endpoints. Precision affects how often a strict sign can be proved, not the sign criterion itself.

The original tree is visited before simplification. Division and negative integer powers require the relevant range to exclude zero strictly; logarithms require positivity. Noninteger constant powers require a nonnegative base for positive exponents and a positive base for negative exponents. Variable powers require a positive base. A square root requires nonnegativity, possibly supplied by Lemma 4.1. Only the stated arithmetic nodes and supported unary functions are admitted. These checks use declared bounds, not a domain inferred from nonlinear constraints, so a convex and feasible model can still be rejected by the sufficient rules.

4.3 Interval cut replay and numerical proposals

The nonlinear expression is translated to SymPy with exact rational leaves, and symbolic derivatives are built from that same expression. The interval evaluator implements addition, multiplication, powers, exponential, logarithm, and absolute value. A rational point is converted by interval division of its integer numerator and denominator, never through a binary64 approximation. Rational powers are evaluated with the applicable square-root or exponential/logarithm formula. If a formula or derivative is unsupported, undefined, or nonfinite at the point, the cut is refused. Recognizing an absolute value or norm as convex does not provide a general nonsmooth derivative rule.

The precise link from differentiation to (9) is a derivative along feasible segments, not merely a list of finite one-sided coordinate derivatives. If, for each $x \in B$, the right derivative of $\theta \mapsto g(z + \theta(x - z))$ at zero is $p^\top(x - z)$, convexity gives

$$g(x) - g(z) \geq \lim_{\theta \downarrow 0} \frac{g(z + \theta(x - z)) - g(z)}{\theta} = p^\top(x - z).$$

A differentiable extension near z is sufficient; valid relative-domain chain rules can also give this condition at a boundary. The symbolic-expression layer is trusted to provide derivatives with this meaning. Finite one-sided coordinate derivatives alone would not suffice: $-\sqrt{xy}$ on the nonnegative quadrant has zero derivatives along both axes at the origin, but the zero vector does not support it. The implemented symbolic quotient derivatives are singular there, so the cut is rejected. Singular derivatives of $-\sqrt{x}$ and unsupported sign expressions at norm origins are likewise refused. Removable singularities may be simplified only consistently with the original checked domain. These are explicit obligations of the expression and differentiation implementation; they do not follow from interval finiteness and are not mechanically verified for every accepted expression.

Cut checking uses a 200-bit interval context. All function and derivative endpoints must be finite. The right-hand side of (11) is enclosed by interval operations, including the maximum of its two endpoint products on a finite coordinate. The final endpoint's sign, integer mantissa, and binary exponent are read directly and converted to a rational, so the endpoint is not rounded through a different precision before it is compared with the supplied intercept. The arithmetic libraries and the symbolic derivative implementation remain trusted components.

The numerical producer uses a separate loaded model. Its OA log supplies candidate row names, points, and slopes; exact reconstructed rows define the certificate mathematics. Points are rationalized and clipped to the propagated box. Slopes are initially shortened to 12 significant

decimal digits by default, with directed repairs on one-sided coordinates and exact slopes where available on free coordinates. The intercept is rounded downward to 30 significant decimal digits after rigorous evaluation. These digit counts control representation size and make replay practically robust; they provide no guaranteed margin between two independent interval evaluations. Replay still requires the exact inequality $b \leq$ the newly computed lower endpoint, with no tolerance. Poor points or wide enclosures can lead to fewer cuts or weaker bounds.

4.4 Binding the discrete proof to the master

The LP writer includes rational affine rows, accepted cut rows, explicit finite bounds, integrality declarations, and the normalized objective. Every free variable has explicit infinite bounds. Fixed variables retain equal bounds, and a nonzero affine objective constant is carried by a new variable `objconst` fixed to one, so the constant cannot be lost silently during proof export. An epigraph coordinate is real and free. Integer bounds tightened by propagation are justified by Lemma 3.1.

The VIPR problem parser is shared with the proof kernel. Comparison requires the same variable set under a bijection, the same integer-variable set, minimization sense, and identical rational objective coefficients. A uniform single `t_` prefix introduced by SCIP can be removed only if this yields that bijection; genuine original names with that prefix are preserved when already matched. Constraints and bounds are compared as multisets after dividing each nonzero row by the absolute value of its first coefficient in sorted variable order. This permits positive rational row scaling with the same relation direction. The matcher is deliberately stricter than linear equivalence: it does not recognize other equivalent row transformations, or negative scaling even of equalities. Additional, missing, or altered rows are rejected, so a certificate for a different presolved model cannot pass merely because it reports the same optimum.

The saved LP must also equal the regenerated LP byte for byte. This is an artifact consistency check; the semantic VIPR comparison supplies the mathematical link to the proof, and a hash or byte comparison alone cannot replace it. Original zero-coefficient affine tautologies are omitted in both constructions, and contradictory constant rows are rejected earlier.

4.5 Proof grammar, arithmetic, and streaming

The kernel accepts complete proofs in a restricted subset of VIPR versions 1.0 and 1.1. It requires the section order

`VER, VAR, INT, OBJ, CON, RTP, SOL, DER`

with the declared numbers of variables, indices, rows, solutions, and derivations. Input uses ASCII integer or fraction tokens with positive denominators; decimal and exponent notation are outside this parser’s contract. Variable names, integer indices, vector coefficient indices, and linear-combination references must satisfy the corresponding uniqueness and range checks, including entries whose value is zero. Omitted solution coordinates mean zero and are still checked against all original rows and integrality conditions. The finite primal endpoint of a range report requires a feasible solution attaining that endpoint or a better one.

Each noncomment derivation occupies one line and carries a complete `asm`, `sol`, `lin`, `rnd`, or `uns` reason implementing Proposition 3.5. Row labels are metadata; references identify rows by indices. The `OBJ` coefficient abbreviation denotes the exact parsed objective. Arithmetic uses rational numbers through python-flint. A combination with incompatible inequality directions or a stronger-than-justified right-hand side is rejected even if its discrepancy is numerically small. Assumptions arise only from actual assumption rows and are tracked through every nonzero inference term.

The checker requires every inference reference to precede its row, including references with zero multipliers. A lifetime annotation is -1 or a nonnegative in-range index; references after a stated last use are errors. The optional single terminal `global` marker emitted by SCIP is accepted only if the independently computed assumption set is empty. It cannot remove assumptions or validate a coefficient. Repeated markers, incomplete reasons, unexpected extra tokens, missing derivations, and noncomment suffix material beyond the declared count are rejected.

An assumption-free row dominating the requested dual endpoint or contradiction is recorded as the proving derivation; it need not be the final row. Every later declared derivation must still pass, and the parser must consume the entire file, so a valid proof prefix followed by invalid arithmetic is rejected. A later valid but weaker bound is harmless. The lower-level kernel can also accept a range with no finite dual endpoint when its remaining obligations hold; the MINLP interface requires a finite lower endpoint before reporting a result.

Large proofs are processed in two passes. The first pass validates record shapes, references, and lifetime annotations, and computes actual last uses with two arrays of eight-byte integers (about 16 bytes per row). The second pass retains one such array and the live sparse rows, freeing each row after its computed final reference; rows annotated -1 are thus also freed when safe. The parser also initially holds the original master rows for solution checking, and this storage and the longest input line count toward memory use. The method avoids storing the whole proof text but has no constant-memory guarantee: many simultaneously live dense rows, long rational tokens, or a large original master can still use substantial memory.

4.6 Failures that motivated the checking boundary

Two small counterexamples show why an external success message is insufficient. The inspected upstream `viprchk` checkout¹ used an initially zero best objective for a solution inference even with an empty solution section. A matching artifact for $\min x$ subject to $x \geq 1$ could therefore be accepted with a claimed lower bound of 10^6 , although $x = 1$ is feasible. Our kernel refuses a solution cutoff without an exactly checked witness. Wood et al. also discuss solution-inference and reference-management weaknesses in older checker behavior [27]; this work makes no priority claim for identifying that general risk.

The second defect, reproduced with the same checkout, rounded the continuous inequality $x \geq 1/2$ to $x \geq 1$. This implication is false, even though every printed coefficient is rational. Our rounding rule checks both integer coefficients and integer variable declarations before applying a floor or ceiling. Likewise, complementary branch inequalities form an exhaustive disjunction only for an integral linear form; adjacent integer right-hand sides are not enough.

Other targeted regressions cover exact floating-leaf aggregation, lost domains after cancellation, singular norm derivatives, omitted slopes on half-lines and free coordinates, objective constants and maximization signs, source/master mismatches, invalid solution witnesses, cross-branch assumption retention, duplicate entries, invalid lifetimes, forged `global` markers, and invalid suffixes after an already sufficient bound. These tests check distinct observable contracts; passing them is not a formal proof of the parser or executable.

Rejection differs from refutation. An insufficient safe-cut enclosure can reject a globally valid cut, and an unsupported curvature rule can reject a convex model. An exact arithmetic audit, in contrast, can establish that a particular recorded inference is invalid; even then the final bound may still hold, since a different valid proof may exist. The experiments therefore report these outcomes separately (Section 6.2) rather than relaxing the check to recover historical success labels.

¹VIPR checkout `30f2951d1e90e47afa821bdd1b12b82246656c42`; the audit records the source hash and regression inputs. The statement concerns those inspected bytes.

4.7 Generation, frozen replay, and trust

The current producer uses numerical OA with Gurobi and IPOPT, followed by exact SCIP and VIPR completion tools. It first tries its default exact-SCIP configuration and can retry with selected separation, presolve, and propagation features disabled. Each attempt has its own search budget, and proof completion and checking add time, so a solver time limit is not a wall-clock limit for the whole pipeline. Fraction canonicalization in newly generated completed proofs preserves their rational values; solution sections and inference meanings are not rewritten to obtain acceptance. Historical artifact directories are preserved.

Replay requires only the checker dependencies (Pyomo, NumPy, SymPy, mpmath, and python-flint) and the included Python modules; checking a complete bundle needs neither Gurobi, IPOPT, nor SCIP. The optional external `viprchk` provides additional corroboration: it must return a zero status and one complete range message matching the exact internal result. The generation harness requests this corroboration; standalone replay need not.

The campaign wrapper hashes model, lemma, LP, and proof files before and after checking. It records source and environment identifiers, input records, optional external-checker identity, explicit missing/rejected/verified/error/timeout outcomes, and the original record index. Resume requires matching artifact fingerprints and the inputs, source/environment identifiers, external checker, and timeout pinned in the manifest; worker parallelism may change. Each worker runs in its own process group, so its timeout also stops descendant checkers. Completed output records are flushed; only an interrupted final JSON fragment may be recovered. These measures provide provenance and detect ordinary changes; they do not establish semantic equivalence or protect against an adversary that changes and restores a file during verification. Stable artifact contents and trusted model execution remain explicit assumptions. Summaries report whether every expected record index is present exactly once and compare objective values exactly, in the objective sense, before converting them for display. An incomplete cohort can still be summarized, with its completeness flag false.

The trusted computing base comprises the loader and exact reconstruction, domain/range and curvature rules, symbolic differentiation, interval arithmetic and endpoint conversion, rational propagation, serialization and master comparison, proof parser and rational kernel, their libraries, and the runtime. The numerical search and proof producer need not be trusted when their outputs pass this contract. Shared trusted code can cause common failures, so separate replay and adversarial tests reduce dependence on the numerical solver without eliminating trust in the implementation. The proof-assistant theorems of the next section cover only their stated mathematical scope; they do not certify this software stack or any benchmark artifact.

5 Proof-assistant verification of the mathematical contract

The certificate interface has a standalone formalization in Lean 4 [8]. Its coverage inventory lists 49 mathematical obligations from Sections 2–4: propagation, supporting cuts, curvature rules, discrete inference, master identity, bound transfer, and the stated mathematical examples. A claim-to-theorem map gives the precise hypotheses and declarations for each obligation. The formalization verifies the mathematical contract and includes executable Lean checkers for typed discrete proofs and rational quadratic matrices. It does not establish that the Python implementation refines those checkers, and no benchmark certificate is checked by Lean.

5.1 Nonlinear cuts and recognition rules

The coordinate type distinguishes bounded intervals, both half-lines, and the real line, so infinite endpoints never enter real arithmetic. The proofs establish the exact suprema in Table 1, including attainment and the necessary and sufficient finiteness conditions, as well as the enclosure tests in Table 2. Rational propagation instructions preserve every mixed-integer feasible point through any finite admitted sequence, including integer rounding, binary intersections, and fixed substitution. Convexity and a justified derivative along each feasible segment imply the supporting inequality. The cut theorem combines that support with genuine value and residual-slope enclosures and the rational intercept test to derive affine underestimation on the whole box. Interval lemmas prove the finite endpoint arithmetic and the sign-dependent rules for unbounded coordinates; they do not verify the execution of an interval library.

The curvature proofs cover the stated composition rules with their domain and sign conditions, quadratic convexity and concavity, PSD homogenization and square-root convexity, real-exponent monomials, and one-variable linear fractions. Quadratic elimination is a structurally recursive rational checker with soundness and completeness proofs for symmetric matrices, including singular and zero-pivot cases. The monomial development proves the actual second derivative in every direction and its displayed Hessian matrix expression, the three exponent criteria, the supporting block identities, and the continuous boundary extensions. These results concern the mathematical functions and checked domains; recognizing their syntax in a loaded expression is a separate implementation obligation.

5.2 Discrete checking and end-to-end transfer

The Lean discrete checker consumes rational rows with equality or either weak inequality, declared integer coordinates, rational solution candidates, and a finite list of typed inference steps. It checks row domination, multiplier signs, integral rounding, and exhaustive integer splits. References address only the checked prefix, assumption sets retain their exact indices, and unsplitting removes each branch’s own assumption without erasing cross-branch dependencies. The theorem `Discrete.check_sound` proves the complete inference invariant from these tests. Every supplied row is checked, including rows after an earlier proving row. Rational solution validation and a proved finite-minimum selection supply the weak incumbent cutoff; `Discrete.checked_bound` then proves the unrestricted master bound. Maximization follows by objective negation, and the separate infeasibility checker uses no solution cutoff.

The end-to-end development constructs the feasible extension rather than assuming its inclusion in the master. It connects admitted propagation steps, accepted supporting-cut data, original affine rows, and integrality to the graph point with its free epigraph coordinate and fixed-one constant coordinate. The affine and epigraph objectives preserve the normalized original objective exactly. A typed master-identity check proves equivalence under variable renaming, integrality correspondence, positive rational row scaling, and complete row permutations, together with objective equality. The resulting bound theorems compose those results with the discrete checker; the sign and infeasibility corollaries have the interpretations stated in Section 3.4.

Primal completion uses an actual feasible nonlinear witness and an objective enclosure. The proofs establish nonemptiness and boundedness before applying the real infimum, derive the bound chain and gap, and prove attainment when the bounds match. Separate extended-real results cover the empty feasible set. The formal examples also check the coefficient-aggregation arithmetic, the complete slope-rounding calculation, boundary-support cautions, invalid inference examples, and domain counterexamples. Kernel-evaluated discrete examples exercise accepted splitting and

rounding, rejected forward references and later invalid rows, retained cross-branch dependencies, and rejected infeasible or fractional incumbents.

5.3 Validation and trust boundary

The package pins Lean 4.33.1 and mathlib 4.33.1 [26], with exact transitive revisions in its manifest. The current development was checked with targeted module builds that treat warnings as failures; we report no result of the project-wide verification script. That script checks complete module imports, builds the project, audits every owned declaration for transitive axioms, and replays compiled declarations through the installed Lean kernel. Its axiom allowlist is `propext`, `Classical.choice`, and `Quot.sound`; the replay is another pass through the same kernel, with imported mathlib as the dependency base. An earlier replay of three modules is kept as a historical record and does not cover the current development.

The remaining trust boundary is operational. Python model loading, expression extraction, symbolic differentiation, actual interval enclosures and transcendental evaluation, serialization, VIPR text parsing, mutable row storage, lifetime accounting, and correspondence between Python execution and the typed Lean checkers are not formally verified. Nor do the proofs verify saved benchmark bundles, external-checker behavior, performance measurements, or novelty claims. The recognition rules are sufficient, not complete for all convex expressions or all valid cuts. Accepted nonlinear evidence must still supply genuine derivative and enclosure data for the same interpreted model.

6 Computational evidence and independent solution audits

The experiments address three questions: how many checkable bounds a frozen uniform protocol produces and how strong they are (Section 6.1); what exact audits establish about failures (Section 6.2); and what checking costs (Section 6.3). Appendix B gives the complete protocol, production accounting, and separately versioned repair results; none of those later checks replaces the frozen uniform outcome.

6.1 Capability: checkable bounds and their strength

A frozen catalogue contains 299 models selected from MINLPLib [21] for convexity and discrete variables. Seven selected files did not load and three did not pass the curvature screen then in use. The remaining 289 names are the complete denominator of the historical and uniform campaigns. The supplement lists all 299 names and the ten exclusions with their recorded reasons. The selection is a snapshot, not a claim about the current size or classification of MINLPLib. We distribute the loaded Python model files with MINLPLib’s CC-BY 4.0 attribution. GAMS is the library’s primary representation; the certificates concern the exact loaded expression semantics of Section 2, not an unverified equivalence of every converted model.

The uniform protocol gives numerical OA and each exact-SCIP attempt a requested 30-second search budget, allows at most a default and a conservative fallback attempt, and caps each complete instance process group at 360 seconds. It uses six workers, followed by a separate execution of the same checker with six workers and a 1,200-second per-record limit. The search budgets exclude additional preprocessing, proof completion, and checking work, which the outer cap covers. All 289 names remain in the denominator, including failed generation.

Separate replay accepts 203 artifacts, rejects 19, and finds 67 without a complete bundle (Table 4). The producer itself returns 198 accepted results. Four further accepted artifacts survive outer

Table 4: Complete replay outcomes and exact reference comparisons. Both denominators are 289; generation outcomes are reported separately in the text. Historical replay also used external corroboration. Reference rows count only verified bounds; d is the signed normalized difference from an unverified reference (Section 6.1).

Quantity	Historical archive	Uniform-run artifacts
Verified	188	203
Near reference: $0 \leq d \leq 10^{-4}$	52	46
Near reference: $0 \leq d \leq 10^{-2}$	81	80
Negative reference difference	23	25
Rejected	92	19
Missing complete artifacts	9	67
Timed out during replay	0	0
Other replay errors	0	0

timeouts and one survives a worker error, so an unsuccessful process return need not destroy a completed checkable proof. The replay count therefore measures the availability of justified bounds under the stated protocol, not only the producer’s return status. It does not establish coverage of all convex MINLPs or superiority over another solver.

Strength relative to recorded objectives. For a replayed bound B in the original objective sense, a recorded library objective r , and $s = 1$ for minimization or $s = -1$ for maximization, we compute

$$d(B, r) = \frac{s(r - B)}{\max\{1, |r|\}}.$$

The bound and the decimal reference string are converted to exact rationals before subtraction. We count bounds with $0 \leq d \leq 10^{-4}$ and with $0 \leq d \leq 10^{-2}$, and count $d < 0$ separately. This is a signed normalized difference from an unverified reference, not an optimality gap; even $d = 0$ does not prove that a feasible nonlinear point exists at that value.

Among the 203 accepted uniform-run bounds, 46 satisfy the first threshold and 80 the second; 25 differences are negative. Some bounds are thus close to recorded objectives, but the success count alone does not imply tight bounds. Among the 188 accepted historical bounds, 52 satisfy the first threshold and 81 the second. The historical comparison has 23 negative differences, all of magnitude below $6 \cdot 10^{-10}$. Such small differences can arise from rounded reference strings, and we do not read their sign as evidence of an incorrect solver result. The supplement records every exact comparison, including those summarized in Table 4.

Bounds versus completed optimality proofs. A close reference value supplies no primal witness. Exact original-model feasibility checks complete the bound for `clay0204m`, giving the optimum 6545 (Section 6.2), and for the small quadratic example below. These are examples of the stronger conclusion available when primal and bound evidence match; we do not infer a benchmark optimality count from objective comparisons.

The core’s solver-free replay includes a mixed-integer quadratic example, $\min y$ subject to $(x - \frac{1}{2})^2 \leq y$, $x \in \{0, 1\}$ and $0 \leq y \leq 2$. Eleven checked cuts and 39 proof derivations establish the lower bound $1/4$, attained by $(x, y) = (0, 1/4)$. The `clay0204m` matching-point audit and the targeted repaired examples exercise larger complete bundles. These checks supplement the 161

targeted software tests, which are not a formal proof of the implementation; Section 5 states the separate proof-assistant result and its boundary.

Beyond the uniform experiment, the retained historical and targeted repair artifacts form a descriptive collection. The repair versions and their separate denominators are specified in Appendix B.3. The descriptive merged catalogue contains verified bounds for 222 distinct loaded models. It chooses the strongest exact normalized lower bound among 405 accepted records from the historical, primary, producer-repair, and reporting-repair evidence, requiring identical model SHA-256 and objective sense for every comparison. Exact ties use a fixed cohort order. Each entry identifies its original-sense bound, complete proof, source cohort, and hashes. This union is a reusable bound collection; its coverage is not attributed to the uniform search budget.

6.2 Failure mechanisms: what rejection establishes

A rejection is informative only when tied to its checking obligation. The uniform run’s 67 admission errors occur at the curvature/domain check, before master generation; they account for all 67 missing replay bundles. Its nineteen rejections comprise eighteen proof-rule or proof-grammar failures and one post-proof reporting failure. The latter concerns the representation of an already checked exact bound, not its mathematical inferences. Separately recorded repairs address that failure and twelve producer normalization failures without changing the proof, domain, curvature, propagation, or master-matching rules (Appendix B.3).

Invalid supplied discrete inferences. Three newly generated completed proofs receive an external acceptance message but fail local conditions of the internal rules. Two linear combinations mix opposite inequality directions: in each, a strictly positive multiplier of an upper branch row appears among lower-row terms. The offending rows have nonzero coefficients and are not constant tautologies. In the third proof, an `uns` step cites the same integer coordinate u with branches $u \leq 64$ and $u \geq 66$, leaving the integer value 65 uncovered. Other master rows might in principle exclude that value, but the submitted disjunction step does not establish such an exclusion. The supplement gives the exact antecedents and multipliers. These findings concern individual supplied inferences and do not imply that the final bounds are false. They show that external acceptance does not discharge the internal inference obligations; the exact row directions and the uncovered integer value are local witnesses, stronger evidence than process status or a small objective discrepancy.

The historical audit shows the effect of enforcing all layers of the contract. Of 269 historical records labeled accepted, 188 retain that verdict. The 81 revocations comprise 67 domain or curvature failures, 13 cut-verification failures, and one discrete-proof failure. Eleven further discrete-proof failures come from historical records already labeled as crashed, giving 92 current rejections in total. Nine records lack a completed proof. No record in the complete historical replay timed out or changed during checking. These counts replace the historical acceptance claim; the old flags are archived only to make the change auditable.

The twelve historical failures that reach the discrete proof checker admit small exact arithmetic audits. In eleven, the claimed upper row has exactly the coefficients of the supplied rational linear combination, but its right-hand side is smaller than the combination permits. The exact positive excess ranges from approximately $1.55 \cdot 10^{-12}$ to $1.93 \cdot 10^{-10}$; an arbitrarily small excess still invalidates the inference over the reals. In the remaining instance, `smallinvDAXr5b200-220`, a positive multiplier uses an upper inequality in a combination requiring lower inequalities, so the multiplied directions are incompatible. The portable supplement includes the twelve first failing steps, their referenced rows, and an independent `Fraction`-arithmetic recombination script; full

completed proofs are in the bulk archive. These are invalid justifications, not proofs that the final proposed bounds are unattainable or false.

Conservative nonlinear failures. Nonlinear rejection has a different meaning. For example, the first historical `risk2bpb` cut exceeds the recomputed sufficient safe intercept by approximately $1.17 \cdot 10^{-12}$. This is a failure of the certificate test, not a proof that the affine inequality is invalid. A separately recorded repaired generation for `risk2bpb` passes all checks and gives an original-sense bound of approximately -55.876139395 . The library string -55.8761394 lies slightly beyond this bound, consistent with rounding at the reference’s displayed precision. Targeted repaired generations for `batchdes`, `clay0204m`, and `risk2bpb` are retained as regression evidence, separate from the uniform campaign.

Returned points and matching primal evidence. Objective disagreements guide investigation; checks in the original variables determine what can be concluded about a returned point. For saved solver objectives r_{solver} with recorded GAMS model statuses 1 (Optimal), 2 (Locally Optimal), or 8 (Integer Solution),² we flag pairs satisfying

$$s(B - r_{\text{solver}}) > 10^{-6} \max\{1, |B|\}.$$

This one-sided criterion flags 18 historical bound/solver pairs as candidates for investigation. The two returned-point audits below support stronger, explicitly limited conclusions.

For `clay0204m`, a saved SBB run reports objective 5075 with normal completion and model status *Integer Solution*; its log also warns of a nonconvex model, so we do not treat it as a claim of global optimality. Its printed point violates the two rows

$$x_2 - x_4 + 51b_{13} \leq 46.5, \quad x_6 - x_7 + 86b_{24} \leq 82$$

by 4.5 and 3, respectively. Allowing every printed coordinate an error of 0.00005 changes the row values by at most 0.00265 and 0.0044, so the violations are robust to the displayed precision. Independently, exact rational evaluation of a 52-variable point checks all 90 loaded constraints and 32 binary variables at objective 6545, and a complete certificate proves the matching lower bound 6545. Together these establish the exact optimum of the loaded model. The archived point comes from rounded BARON output with small distance coordinates replaced by zero; its feasibility follows from the new rational evaluation, not from BARON’s status or tolerance.

For `risk2bpb`, the saved SHOT run reports an objective near -56.82087261 and optimal model status. Its printed point assigns one to both b_4 and b_5 , although both variables are fixed at zero in the source model. These unit violations suffice to reject the returned point, but they do not identify the internal cause of the discrepancy. The archived run used SHOT 1.1 with CPLEX 22.1; the baseline scripts requested 60 seconds, relative and absolute optimality tolerances 10^{-4} and 10^{-6} , and four threads.

To ensure that these statements compare the intended formulas, a separate source audit matches the GAMS and loaded Python models under the same exact binary64-constant interpretation. It eliminates the GAMS free objective variable and its defining equality, then compares each remaining constraint, variable type, fixed value, and bound under an explicit index map. The `clay0204m` comparison maps 53 variables and 91 rows to 52 variables and 90 rows; `risk2bpb` maps 464 variables and 581 rows to 463 variables and 580 rows. The violated rows and fixed bounds use exactly representable coefficients, so the violations do not depend on the distinction between decimal and

²GAMS 53 User’s Guide, [Model Status table](#).

binary64 constants. This establishes source-formula equivalence; it does not verify the GAMS compiler or recover either solver’s historical in-memory model. The conclusions therefore concern the saved returned points and the specified source models; we attribute no unobserved internal defect, and neither conclusion treats a solver status or library objective as a primal certificate.

6.3 Costs and practical limits

Replay avoids repeating numerical search, but it does not make large certificates cheap to store or check. The uniform campaign takes 2,840.697 seconds of elapsed generation time, and its later separate replay takes 533.541 seconds. Accepted checker calls sum to 2,791.064 seconds, with median 5.173 and maximum 221.484; all 289 records sum to 2,953.982 seconds. These are observed shared-machine costs with concurrent workers, not serial elapsed times or controlled speed comparisons. Complete checking includes nonlinear and model checks, master matching, and proof replay, not only discrete inference arithmetic. Historical replay also includes external corroboration; its costs are reported separately in Appendix B.

The 203 accepted uniform proofs occupy 14,233,742,052 bytes and contain 11,749,855 derivations. Historical proof sizes and the complete proof collection are reported in Appendices A and B. These figures show a practical storage burden; they are not a certificate-size comparison with prior methods.

The two-pass replay of Section 4 does not retain the whole proof text and frees sparse rows after their last use. Its storage still includes per-derivation index arrays and depends on the original master, input-line length, rational sizes, and simultaneously live rows. The machine’s memory capacity (Appendix B) is not a measurement or bound on checker peak memory, and we infer no memory or speed advantage from these runs.

Running the checker does not require Gurobi, IPOPT, SCIP, or GAMS. License-identifying solver banners are removed only from distributed log copies; a redaction manifest records original and distributed hashes. Model files, mathematical data, and proof files are unmodified.

7 Discussion and conclusions

The certificate interface makes the implication from a rational master bound to a convex MINLP bound explicit and checkable. Its core is objective-preserving feasible-set inclusion: rigorously justified cuts retain every nonlinear feasible point, master identity binds those cuts to the discrete proof, and complete rational replay establishes the transferred bound. Numerical search can therefore propose evidence without its tolerances becoming part of the final mathematical assertion.

First, exactness must refer to one model throughout the calculation. Exact arithmetic in the discrete proof is insufficient if curvature analysis, derivative evaluation, or master construction interpret an expression differently. The loaded-tree contract fixes the coefficients and domains to which the certificate refers, so the bound concerns that exact model; relating it to an earlier modeling file is a further semantic obligation.

Second, verification failures need mathematical classification. The audits identify invalid supplied linear combinations and integer disjunctions, including steps that received external acceptance; tiny residuals and successful process status do not justify those steps. Other rejected artifacts fail sufficient domain, curvature, or cut-enclosure tests and may still represent convex models or valid cuts. A local invalid inference refutes its supplied justification, not necessarily its final proposed bound. Retaining the evidence makes these distinctions reproducible.

Third, primal feasibility supplies information that objective comparisons cannot. Original-variable audits expose violations in two saved solver points, while independently checked original-model witnesses complete exact optimality proofs in two cases. These conclusions require checking

bounds, integrality, rows, and the objective for the same model; a feasible master point or a close library reference value cannot substitute for that work.

The uniform experiment yields accepted artifacts for 203 of 289 attempted models in separate replay, against 198 successful producer returns under the same protocol. Complete proofs that survive unsuccessful producer returns show the value of separating an artifact’s validity from a process outcome. The long-rational failures show that exact values must survive text conversion and reporting as well as inference arithmetic. The incomplete admission rules, large proof files, and shared-machine timings limit broader conclusions about coverage and efficiency. Two-pass replay reduces the proof rows retained in memory without bounding the number of simultaneously live rows, and the measurements do not establish a speed advantage over other certificate methods or solvers.

Executable tests and exact local audits exercise concrete checking behavior. Lean verifies the mathematical contract and typed reference checkers, but not the Python loader, symbolic and interval software, master matcher, or VIPR checker, nor their correspondence to the Lean definitions (Section 5). The result is a replayable certificate method for the stated model class with an explicit trust boundary, supported by reproducible evidence of its capability and reliability.

A Data, code, and reproduction

The accompanying materials suffice for reading the paper, building its PDF, checking the Lean development, and replaying the reported experimental evidence. They consist of three separate archives; no public deposit or DOI is claimed. The small core and bulk proof archives are identified by byte counts and SHA-256 hashes in `supplement/archives.json`, which is included with the paper source. MINLPLib inputs retain their attribution and license notice. Installed third-party packages, licensed solver binaries, and user-supplied literature PDFs are not redistributed.

Paper and formal sources. The archive `certified-minlp-paper-source.tar.gz` contains `main.tex`, all required sections and generated tables, the selected bibliography, the vector figure source, and the standalone `formal/` project with its dependency pins, coverage statement, and audit scripts. Extract it and run, from the extracted paper directory,

```
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

A TeX installation with BibTeX and the packages listed in `main.tex`, including TikZ, is sufficient; no experimental archive is needed. The source archive contains no stale LaTeX auxiliary files or Lean caches. To check the Lean development, install Elan, Git, and Python 3 and run from `formal/`:

```
sha256sum -c verification/extension-SHA256SUMS
lake exe cache get
bash scripts/verify.sh
```

The toolchain and manifest select the exact Lean and mathlib versions. Downloading the dependency cache is separate from building and replaying the project proofs. The saved three-module replay, which audited 95 declarations, covers only the earlier scope; the current development has separately recorded targeted module checks, and no result of the full script is reported here. Section 5 and the coverage map specify the current mathematical scope and its implementation boundary.

Checker-only core. The archive `certified-minlp-core.tar.gz` is 6,478,681 bytes compressed. It contains all 289 campaign model files, current checker code, pinned dependency specifications, versioned source snapshots, compact records and audit data, representative complete bundles, and the scripts used to regenerate tables and the bound catalogue. Extract it to obtain `minlp-certified-evidence/`. From that directory, using Python 3.13:

```
python3 -m venv replay-env
replay-env/bin/python -m pip install -r requirements-checker.txt
replay-env/bin/python reproduce.py verify
replay-env/bin/python reproduce.py small --out ../small-checks
replay-env/bin/python reproduce.py summaries --out ../summary.json
```

The five checker dependencies are Pyomo, NumPy, SymPy, mpmath, and python-flint. The `small` command checks four complete bundles, the exact original-model source and primal audits, and fifteen local failed-step extracts. The 161-test suite additionally needs `pytest`, as specified in `requirements-tests.txt`. None of these commands needs licensed optimization software. The core README gives commands that regenerate all thirteen table and catalogue outputs from saved records; this reproduces recorded results, not new solver timings.

Complete proof collection. The separate archive `certified-minlp-certificates.tar.gz` is 30,664,561,063 bytes compressed and contains 93,713,729,595 bytes of regular-file contents. It retains accepted and rejected completed proofs and completed fallback attempts. For full replay, extract it into the same parent directory as the core. The core provides a streaming archive verifier and commands for complete historical and uniform replay and for the separate repair cohorts. A full readback matched all 5,207 archive entries to their recorded file hashes or symlink targets; routine example checking does not require repeating it.

The default distributed checker is version V3; Appendix B.3 gives the measured V1 outcome and the expected V3 outcome. The core documents restoration of V1, V2, and V3 into separate copies, including shared test and example data. Their restored test counts are 152, 153, and 161, respectively. New replay commands write new records and manifests.

Generation and provenance. Appendix B specifies the frozen experimental protocols, full production accounting, and separate repair cohorts. Generating new evidence requires the numerical solvers and proof-production tools described in the core README: Gurobi, IPOPT, exact SCIP, and VIPR completion and checking utilities. The wrapper accepts local executable paths and freezes models, software, settings, budgets, and environment before launch. Absolute paths in frozen manifests document the recorded machine; portable replay resolves files relative to the extracted layout. Numerical generation and elapsed times can change with machine load or software builds; exact strings, input hashes, versioned code, and complete outcome records define the reproducible checking claim.

B Experimental protocol and supporting accounting

This appendix records the frozen protocols and the full production and repair accounting behind Section 6. Its summaries are derived from the frozen campaign records, with separate denominators for the primary and targeted runs.

B.1 Frozen protocols and measurement boundaries

The historical campaign used a 60-second numerical outer-approximation budget, heterogeneous exact-SCIP budgets of 180 or 90 seconds, and varying concurrency. Its archived producer labels are therefore neither current acceptance evidence nor results of a uniform timing experiment. We replayed all 289 records with six workers and a 1,200-second per-record limit, including both the internal exact proof kernel and the external `viprchk` corroboration. The replay manifest identifies the checker sources and inputs by SHA-256; the harness checks artifact hashes again after checking.

We also generated new artifacts for the same 289 names, including every historical failure, using a protocol frozen before the first run. Numerical outer approximation and each exact-SCIP attempt received a requested search budget of 30 seconds. The producer tried its default configuration and, after an unsuccessful complete check, at most one conservative fallback. We requested one solver and LP thread and ran six workers. A separate 360-second wall-clock limit killed the entire instance process group, retaining partial files and recording the timeout without retry. The 30-second settings are search budgets: loading, preprocessing, nonlinear subproblem overruns, proof completion, and checking are additional work covered by the outer limit. Tool events distinguish SCIP invocations, proof completion, external checking, and internal proof replay.

After generation, a separate complete replay used six workers and a 1,200-second per-record limit. Acceptance is decided by a separate execution of the same internal checker, which is implemented separately from the solver and the external VIPR checker; the generator’s external-check results are additional evidence. The replay chooses the canonical completed proof when present, otherwise the completed failed fallback, otherwise the completed failed default attempt. This rule was fixed before replay and uses no bound-quality selection. Consequently, a complete proof surviving a producer timeout can earn a separate replay verdict without being counted as completed production. All attempt outcomes remain in the data. We rehash the frozen sources, settings, model inputs, wrapper, and tool executables after generation.

The machine has an Intel Xeon w5-2565X processor (18 physical cores, 36 logical processors) and approximately 30 GiB of memory, running Linux under WSL2. Generation uses Gurobi 13.0.3, IPOPT 3.14.20, and SCIP 10.0.3 with SoPlex 8.0.3. Package versions, executable hashes, and the full hardware description are recorded with the protocol. The one-thread setting concerns OA, SCIP, and LP search; proof completion keeps its tool defaults (its banner reports 36 available threads), so the pipeline as a whole is not single-threaded. Times are observed costs on a shared machine, not isolated solver speed comparisons, and sums over concurrent workers are not campaign elapsed times. Large archive compression takes place after the timed campaigns.

B.2 Complete production and replay accounting

The following generated summary retains every primary V1 production outcome, including process failures whose surviving artifacts later verify. It also records the selected proof sizes, derivation counts, and checker timings. The uniform generation records 198 completed accepted productions, 67 producer errors, 8 unsuccessful completed-check attempts, 4 outer timeouts, and 12 worker errors. Its elapsed time is 2,840.697 seconds. Separate replay accepts 203 bounds, rejects 19 bundles, and finds 67 incomplete bundles. Of its accepted bounds, 4 come from outer-timed-out production, 1 from worker errors, and 0 from producer rejections; 0 use a retained failed-attempt completed proof. Accepted proofs occupy 14,233,742,052 bytes and contain 11,749,855 derivations. Their summed and median checking costs are 2,791.064 and 5.173 seconds, respectively; the maximum is 221.484 seconds. All-record checking costs sum to 2,953.982 seconds and separate replay elapsed time is 533.541 seconds. These observations characterize this fixed protocol, including its unsuccessful

Table 5: Observed in-generation calls in the uniform campaign. Sums include completed calls only; starts without ends are interrupted observations. SCIP times cover the process invocation, not just search. These rows are separate from full pipeline and later replay times.

Phase	Started	Ended	Seconds (sum)
SCIP process	358	358	7,401.357
Proof completion	358	358	371.307
External proof check	477	477	1,290.663
Internal proof replay	202	201	1,583.153

attempts; they do not estimate asymptotic coverage or establish superiority over another solver.

Section 6.2 classifies the 67 producer errors, all at domain/curvature admission, and the nineteen primary replay rejections. Table 5 separates recorded tool phases during generation; interrupted calls have no imputed completion time.

Under the historical replay protocol, the accepted proofs contain 29,903,993 derivations and occupy 38,826,726,525 bytes. Their median size is 74,469,242 bytes and their largest size is 2,163,285,624 bytes. Median accepted checking time is 16.679 seconds, with a maximum of 463.845 seconds. The sum over accepted records is 7,987.954 seconds; the sum over all 289 records is 8,234.349 seconds. The campaign elapsed time is 1,465.381 seconds. These checking times include model and lemma validation, master reconstruction, and internal and external proof checking (hashing is recorded separately), so the aggregate does not measure the linear-proof kernel alone.

B.3 Separately versioned representation repairs

V1 denotes the frozen primary producer and checker. V2 repairs producer proof normalization, and V3 additionally repairs exact rational report input/output. These changes concern the survival of exact values through textual interfaces, not the mathematical proof rules. Keeping all three versions separates the uniform protocol result from targeted recovery of affected artifacts.

The uniform run exposed twelve worker failures when proof normalization passed a rational token through Python’s decimal-to-integer conversion limit. We preserved this campaign and its producer source, then replaced that conversion with parsing and serialization through the existing FLINT rational type. A separate cohort regenerated all twelve observed occurrences under the same requested search budgets, settings, concurrency, and outer limit. The original 289-record denominator and its failures remain unchanged.

A further reporting boundary appeared in `tls12`: its primary bundle passed nonlinear checks, exact master matching, and the entire proof kernel, but a rational bound with a 4,326-digit numerator failed during subsequent report construction. The same boundary appeared in the default attempt of the producer-repair cohort. A third source version repairs exact rational text input and output at the reporting interfaces, including replay comparisons and reference summaries. Decimal reference strings retain exact decimal semantics; the interpreter’s global integer-conversion guard remains enabled. Optional binary64 display values become unavailable when the exact value is outside binary64 range, while exact strings remain authoritative. The proof, domain, curvature, propagation, and master-matching rules are unchanged.

All twelve producer-repair runs returned verified output, and all twelve selected bundles passed separate replay. Generation took 478.703 seconds of elapsed time and replay took 62.473 seconds. Auditing all nineteen primary rejections and all secondary attempt reports identified exactly two completed proofs affected by the subsequent reporting defect, both for `tls12`. With the third source

version, both passed full replay in a separate 73.431-second cohort, without further optimization. The frozen primary replay counts remain 203 verified and 19 rejected; the corrected reporting layer accepts one additional primary bundle. The supplement preserves the sources, records, and completed attempts for each version. These targeted repairs do not constitute a second uniform experiment or replace its denominator.

The current checker is V3. Its two targeted checks establish one additional accepted primary artifact; they are not a new full 289-record replay. A complete V3 replay of the fixed primary artifacts is expected to give 204 verified, 18 rejected, and 67 missing, subject to the documented time and environment limits; the measured uniform outcome remains V1’s 203/19/67 (Appendix A gives the restoration and replay commands). The descriptive catalogue combines protocols by matching model hashes and objective senses; its 222-model coverage is not attributed to a single search budget.

References

- [1] Michel Baes, Timm Oertel, and Robert Weismantel. Duality for mixed-integer convex minimization. *Mathematical Programming*, 158(1–2):547–564, 2016. doi: 10.1007/s10107-015-0917-y. URL <https://doi.org/10.1007/s10107-015-0917-y>.
- [2] Amitabh Basu, Michele Conforti, Gérard Cornuéjols, Robert Weismantel, and Stefan Weltge. Optimality certificates for convex minimization and Helly numbers. *Operations Research Letters*, 45(6):671–674, 2017. doi: 10.1016/j.orl.2017.10.002. URL <https://doi.org/10.1016/j.orl.2017.10.002>.
- [3] Alexander Bentkamp, Ramon Fernández Mir, and Jeremy Avigad. Verified reductions for optimization. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 13994 of *Lecture Notes in Computer Science*, pages 74–92. Springer, 2023. doi: 10.1007/978-3-031-30820-8_8. URL https://doi.org/10.1007/978-3-031-30820-8_8.
- [4] Sander Borst, Leon Eifler, and Ambros Gleixner. Certified Constraint Propagation and Dual Proof Analysis in a Numerically Exact MIP Solver, 2024. URL <https://arxiv.org/abs/2403.13567>.
- [5] CakeML Project. Formalisation of VIPR: Verifying integer programming results. Software and HOL4 proof development, 2026. URL <https://github.com/CakeML/cakeml/tree/8a32c2e8a6a3cfc7495ed13c8de9865df0c15510/examples/vipr>. Inspected commit 8a32c2e8a6a3cfc7495ed13c8de9865df0c15510; accessed September 13, 2026.
- [6] Kevin K. H. Cheung, Ambros Gleixner, and Daniel E. Steffy. Verifying integer programming results. In *Integer Programming and Combinatorial Optimization*, volume 10328 of *Lecture Notes in Computer Science*, pages 148–160, 2017. doi: 10.1007/978-3-319-59250-3_13.
- [7] Chris Coey, Miles Lubin, and Juan Pablo Vielma. Outer approximation with conic certificates for mixed-integer convex problems. *Mathematical Programming Computation*, 12:249–293, 2020. doi: 10.1007/s12532-020-00178-3. URL <https://doi.org/10.1007/s12532-020-00178-3>.
- [8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.

- [9] Marco A. Duran and Ignacio E. Grossmann. An outer-approximation algorithm for a class of mixed-integer nonlinear programs. *Mathematical Programming*, 36(3):307–339, 1986. doi: 10.1007/bf02592064. URL <https://doi.org/10.1007/BF02592064>.
- [10] Leon Eifler and Ambros Gleixner. Safe and Verified Gomory Mixed-Integer Cuts in a Rational Mixed-Integer Program Framework. *SIAM Journal on Optimization*, 34(1):742–763, 2024. doi: 10.1137/23m156046x. URL <https://doi.org/10.1137/23M156046X>.
- [11] Sourour Elloumi, Amélie Lambert, Bertrand Neveu, and Gilles Trombettoni. Global solution of quadratic problems using interval methods and convex relaxations. *Journal of Global Optimization*, 91(2):331–353, 2025. doi: 10.1007/s10898-024-01370-8. URL <https://doi.org/10.1007/s10898-024-01370-8>.
- [12] Michael Grant, Stephen Boyd, and Yinyu Ye. Disciplined convex programming. In Leo Liberti and Nelson Maculan, editors, *Global Optimization: From Theory to Implementation*, Nonconvex Optimization and Its Applications, pages 155–210. Springer, 2006. URL https://stanford.edu/~boyd/papers/disc_cvx_prog.html.
- [13] Katrin Halbig, Lukas Hümbes, Florian Rösel, Lars Schewe, and Dieter Weninger. Computing Optimality Certificates for Convex Mixed-Integer Nonlinear Problems. *INFORMS Journal on Computing*, 36(6):1579–1610, 2024. doi: 10.1287/ijoc.2022.0099. URL <https://doi.org/10.1287/ijoc.2022.0099>.
- [14] Christopher Hojny, Mathieu Besançon, Ksenia Bestuzheva, Sander Borst, João Dionísio, Johannes Ehls, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, Adrian Göß, Alexander Hoen, Jacob von Holly-Ponientzietz, Rolf van der Hulst, Dominik Kamp, Thorsten Koch, Kevin Kofler, Jurgen Lentz, Marco Lübbecke, Stephen J. Maher, Paul Matti Meinhold, Gioni Mexi, Til Mohr, Erik Mühmer, Krunal Kishor Patel, Marc E. Pfetsch, Sebastian Pokutta, Chantal Reinartz Groba, Felipe Serrano, Yuji Shinano, Mark Turner, Stefan Vigerske, Matthias Walter, Dieter Weninger, and Liding Xu. The SCIP Optimization Suite 10.0, 2025. URL <https://doi.org/10.48550/arXiv.2511.18580>.
- [15] Christian Jansson. A rigorous lower bound for the optimal value of convex optimization problems. *Journal of Global Optimization*, 28(1):121–137, 2004. doi: 10.1023/B:JOGO.00000006720.68398.8c. URL <https://doi.org/10.1023/B:JOGO.00000006720.68398.8c>.
- [16] Christian Jansson. On verified numerical computations in convex programming. *Japan Journal of Industrial and Applied Mathematics*, 26(2–3):337–363, 2009. doi: 10.1007/BF03186539. URL <https://doi.org/10.1007/BF03186539>.
- [17] Jan Kronqvist, David E. Bernal, Andreas Lundell, and Ignacio E. Grossmann. A review and comparison of solvers for convex MINLP. *Optimization and Engineering*, 20:397–455, 2019. doi: 10.1007/s11081-018-9411-8. URL <https://doi.org/10.1007/s11081-018-9411-8>.
- [18] Andreas Lundell and Tapio Westerlund. Convex underestimation strategies for signomial functions. *Optimization Methods & Software*, 24(4–5):505–522, 2009. doi: 10.1080/10556780802702278. URL <https://doi.org/10.1080/10556780802702278>.
- [19] Victor Magron, Xavier Allamigeon, Stéphane Gaubert, and Benjamin Werner. Formal proofs for nonlinear optimization. *Journal of Formalized Reasoning*, 8(1):1–24, 2015. doi: 10.6092/issn.1972-5787/4319.

- [20] Frédéric Messine and Gilles Trombettoni. Reliable bounds for convex relaxation in interval global optimization codes. *AIP Conference Proceedings*, 2070:020050, 2019. doi: 10.1063/1.5090017. URL <https://doi.org/10.1063/1.5090017>.
- [21] MINLPLib. MINLPLib: A library of mixed-integer and continuous nonlinear programming instances, 2026. URL <https://www.minlplib.org/>. Accessed September 13, 2026.
- [22] Anthony Narkawicz and César Muñoz. A Formally Verified Generic Branching Algorithm for Global Optimization. In *Verified Software: Theories, Tools, Experiments*, volume 8164 of *Lecture Notes in Computer Science*, pages 326–343. Springer, 2014. doi: 10.1007/978-3-642-54108-7_17. URL https://doi.org/10.1007/978-3-642-54108-7_17.
- [23] Arnold Neumaier and Oleg Shcherbina. Safe bounds in linear and mixed-integer linear programming. *Mathematical Programming*, 99(2):283–296, 2004. doi: 10.1007/s10107-003-0433-3. URL <https://doi.org/10.1007/s10107-003-0433-3>.
- [24] Alexey Solovyev and Thomas C. Hales. Formal Verification of Nonlinear Inequalities with Taylor Interval Approximations. In *NASA Formal Methods*, volume 7871 of *Lecture Notes in Computer Science*, pages 383–397. Springer, 2013. doi: 10.1007/978-3-642-38088-4_26. URL https://doi.org/10.1007/978-3-642-38088-4_26.
- [25] Stefan Szeider. VIPR certificate construction from black-box ILP solvers. In *32nd International Conference on Principles and Practice of Constraint Programming (CP 2026)*, volume 379 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 52:1–52:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026. doi: 10.4230/LIPIcs.CP.2026.52. URL <https://doi.org/10.4230/LIPIcs.CP.2026.52>.
- [26] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.
- [27] Kenan Wood, Runtian Zhou, Haoze Wu, Hammurabi Mendes, and Jonad Pulaj. Satisfiability modulo theories for verifying MILP certificates. *Journal of Symbolic Computation*, 135:102543, 2026. doi: 10.1016/j.jsc.2025.102543. URL <https://doi.org/10.1016/j.jsc.2025.102543>. Open manuscript: arXiv:2312.10420v4.